01 / EL PROCESO

EL HARNESS DETRÁS DE LA APP.

Cuatro semanas. Cero líneas de producto a mano.
Cómo se organizó el trabajo para lograrlo.

BAHÍA BLANCA, ARGENTINA

19 SEP — 18 OCT 2026



LADO A · EL PROCESO · HOY

LADO B · LA APP · DEMO

CONTEXTO → AGENTES → EVIDENCIA

02 / EL PUNTO DE PARTIDA

ANTES DE GENERAR, ELEGIR BIEN EL PROBLEMA.

Una jam de blues mensual en Bahía Blanca, con 20 a 60 músicos.
Antes de elegirla se evaluaron ocho dominios.

> Un problema verificable hace posible un proceso verificable.

01 La IA tiene que decidir, no resumir.

Doce temas que arranquen arriba, aflojen en el medio, cierren fuerte y no repitan la jam anterior.

02 El resultado se verifica de un vistazo.

Cualquier músico lee la lista y sabe si sirve.

03 La tarea manual es realmente tediosa.

Hoy depende de la memoria de una persona.

04 Un problema propio, con usuarios reales.

PRIMERO DECIDIR. DESPUÉS GENERAR.

01

INVESTIGAR

NotebookLM + documentación oficial

10 APIs

Ninguna da tonalidad ni tempo confiables: el dato central de una jam.

Decisión: la tonalidad la pone el admin.

02

ACOTAR

Conversación de diseño con Claude

3 decisiones

Una fuente por dato. Solo el catálogo propio. Lo que hace el admin, también el asistente.

De Manal o Memphis la Blusera, el modelo sabe poco y alucina.

03

DOCUMENTAR

build-brief + Stitch

7 documentos

Dominio, acceso, diseño, stack y riesgos, escritos para un agente sin contexto.

Stitch inventó un bar y un admin. Lo atrapó contrastar con lo decidido.

04

PLANIFICAR

harness-starter

34 rebanadas

Rebanada: una funcionalidad chica, de punta a punta. Ordenadas por dependencias reales.

8 podían arrancar ese mismo día, sin esperar la planilla.

El descubrimiento se convirtió en contratos para un agente sin contexto.

EL AGENTE NO RECUERDA. EL REPOSITORIO SÍ.

01 / CONTEXTO

AGENTS.md + docs

Un índice corto: reglas con su motivo y qué leer en cada caso.

02 / CONOCIMIENTO

Skills versionadas

El workflow del curso y una skill propia de arquitectura.

03 / ROLES

Tres subagentes

Planificar, implementar y validar con fronteras explícitas.



04 / VERIFICACIÓN

init.sh

Un gate que corre todo y declara qué herramientas corren de verdad.

05 / REGLAS EJECUTABLES

Konsist

Las decisiones de arquitectura se convierten en tests.

06 / ESTADO DURABLE

Plan + progreso + evidencia

Qué está aceptado, qué falta y con qué se comprobó.

CLAUDE CODE — UN MISMO HARNESS — CODEX

QUIEN ESCRIBE NO ES QUIEN ACEPTA.

planner

01 PLANIFICA

Lee el repo, prueba supuestos y define el contrato.

`docs/specs/<id>.md`

→

implementer

02 IMPLEMENTA

Escribe código, corre el gate y registra evidencia.

`Estado: passing`

→

validator · no puede editar

03 VALIDA

Contrasta comportamiento, spec y pruebas reales.

`accept / revise / block`

↰ revise: vuelve a implementación

accept → el orquestador lo registra y commitea

passing ≠ accepted

Autoverificarse no alcanza: las dependencias se destraban solo con aceptación.

9 rebanadas volvieron con `revise` al menos una vez.

UN CASO — El documento de diseño decía que un cupo libre y uno ocupado se distinguían por el relleno. El validador midió la captura en escala de grises: 50 contra 53. No se distinguían.

06 / VERIFICACIÓN

UNA REGLA QUE NO PUEDE FALLAR NO PRUEBA NADA.

• • • VERIFICACIÓN / init.sh

```
$  CI=true ./init.sh

Build → Tests → Lint → Arquitectura

konsist                          wired ✓
detekt                           wired ✓
ktlint                           wired ✓

Al principio decía NOT WIRED YET,
porque era la verdad.
```

1.156
tests JVM en la última corrida

22
reglas ejecutables de arquitectura

PLANTAR UNA VIOLACIÓN
↓
VER EL GATE EN ROJO
↓
REPARAR Y RECIÉN AHÍ CONFIAR

Lo mismo con los tests de producto: cambiar el «cualquiera» del filtro por «todos» tenía que poner algo en rojo. Más de una vez, no pasó.

REGLA DEL GATE Solo se volvió más estricto, nunca más laxo: sin baselines que escondan problemas ni reglas apagadas para que pase.

07 / EL HUMANO EN EL CIRCUITO

LA AUTONOMÍA CONCENTRA EL CRITERIO HUMANO.

01 / DÓNDE FRENAR

Solo donde una decisión cambia el proyecto

Una versión alfa en el gate, subir Kotlin para usar Koog, una regla de arquitectura nueva o contradecir una decisión escrita.

02 / PREGUNTAS CONCRETAS

El producto aparece al preguntar

El Instagram mostraba una jam los martes: era el programa de radio. Resuelto por el agente, la app mostraría una jam que no existe.

03 / VERIFICACIÓN

Sacar al humano de la verificación, no de las decisiones

Jam de demo y sesión de admin solo en debug. El validador abrió el APK de release y comprobó que no están.

04 / LÍMITES

Lo que no está en el prompt, no existe

Un subagente activó TalkBack: la restricción estaba en la memoria del orquestador, no en su prompt. Desde entonces, cada prompt repite los límites.

LOS ERRORES, Y QUÉ LOS ATRAPÓ.

ERROR	LO ATRAPÓ	QUÉ CAMBIÓ
Stitch inventó un bar, un admin y la arquitectura	contrastar con las decisiones escritas	los hechos viven en archivos con su fuente
El gate decía `wired` sin correr nada	el planner, al prototipar	una herramienta cuenta solo si corre en todos los módulos
Tests que seguían en verde con la regla invertida	validator	romper a propósito es parte de la definición de terminado
Un `git commit -a` se llevó cambios a medio hacer de un agente	el build roto en dos commits	commits por ruta explícita mientras corre un agente
Tras pegar el backend, seguía respondiendo la versión vieja	un probe que no escribe nada	todo chequeo se diseña para ser observable desde afuera

Ningún error terminó en «tener más cuidado». Cada uno dejó un cambio en el proceso.

09 / LA PRUEBA DEL PLAN: FASE 2

LA IA ENTRÓ A LA APP SIN TOCAR LAS FEATURES.

Un asistente que le propone la lista al admin.
Diseñado para encajar desde la primera semana.

01 **LEE DATOS REALES**
Catálogo, jams pasadas y agenda, con herramientas de solo lectura.

↓

02 **ELIGE, ORDENA Y EXPLICA**
Una propuesta estructurada, que llega tema por tema.

↓

03 **LA APP VALIDA**
Lo que no está en el catálogo, se descarta.

↓

04 **EL ADMIN DECIDE**
Aplica con las mismas funciones que usa a mano.

0 módulos de feature existentes tocados al agregar el asistente.

Konsist lo comprueba en cada corrida: el agente no puede escribir y la feature solo aplica a través del contrato.

SOLO APARECIÓ AL EJECUTAR

- El gateway aceptaba herramientas, pero el modelo no las usaba. La spec ya traía el plan B.
- El gateway se cayó con 502. La rebanada quedó blocked, no passing: no se verifica lo que no corre.

VALIDAR POR REBANADA NO ALCANZA.

9 hallazgos de un code review de todo el repo

7 rebanadas nuevas, con el mismo planner → implementer → validator

3 hallazgos convertidos en reglas de Konsist, para que no vuelvan

11 → 1

copias del mismo mecanismo de escritura, unificadas en un motor.

122 → 14

declaraciones públicas en las features: solo quedan las pantallas.

2

roturas del motor nuevo que los tests viejos no detectaban. Se agregó un test antes de aceptar.

EL HARNESS IMPORTA MÁS QUE EL PROMPT.

01 Separar roles encuentra lo que un solo agente no ve.

02 Una regla que no puede fallar no prueba nada.

03 Los subagentes no heredan contexto: lo que no se escribe, no existe.

04 Acotar el LLM a datos propios es lo que lo vuelve útil.



4 semanas	29 decisiones humanas	1.156 tests JVM	+240 commits	0 líneas de producto escritas a mano

La IA multiplica el criterio.
El criterio sigue siendo nuestro.